Nexcon Building Management

Website & Booking System - Developer Handoff Guide

This document is a complete reference for the nexconbm.com.au website: how it's built, how each moving part works, and what to know before changing anything. It's written for whoever picks this project up next.

Live site: https://nexconbm.com.au

Repository: github.com/nexconBM/NexconBM (branch: main)

Hosting: GitHub Pages, custom domain via the CNAME file

Stack: plain HTML/CSS/JavaScript - no framework, no build step beyond one small Python templating script. No server of its own; a Google Apps Script "Web App" acts as the only backend, for the inspection booking feature.

Document generated: October 2026

Contents

1. Site overview and repository layout

2. The build system (build.py)

3. Bilingual support (translations.js / i18n.js)

4. Styling and the cache-busting convention

5. Contact form (enquiry.js + FormSubmit)

6. Case studies from a Google Sheet (case-studies.js)

7. Inspection booking system (booking.js + Apps Script + Google Sheet)

8. Security posture and known gaps

9. Deployment and day-to-day workflow

10. Known loose ends

Appendix A: full Apps Script backend source

1. Site overview and repository layout

The site is a set of static HTML pages, one per folder, each with its own `index.html`. There is no React/Vue/build tooling: every page is plain HTML with a shared header and footer stitched in by a small Python script (`build.py`), shared CSS (`styles.css`), and a handful of small, single-purpose JS files loaded where needed.

Top-level files

- `index.html` - homepage
- `styles.css` - the only stylesheet, shared by every page
- `translations.js` - English/Chinese text dictionary (see section 3)
- `i18n.js` - applies the language toggle
- `enquiry.js` - powers the contact/enquiry forms (section 5)
- `case-studies.js` - pulls extra project case studies from a Google Sheet (section 6)
- `booking.js` - the inspection booking calendar widget (section 7)
- `build.py` - the header/footer templating script (section 2)
- `partial/header.html`, `partial/footer.html` - the single shared copies build.py stitches into every page
- `CNAME` - custom domain for GitHub Pages (currently `nexconbm.com.au`)
- `robots.txt`, `sitemap.xml` - SEO/crawler config
- `404.html` - custom not-found page

Page folders

Each real page lives in its own folder as `<name>/index.html`, so links are clean (e.g. `/custom-homes/`). The current live pages, listed in `build.py`'s `PAGES` list, are:

- index.html (home), about, contact, custom-homes, faq, house-and-land, building-inspections, knockdown-rebuild, multi-unit-development, projects, project-management, renovations-extensions

> **Note:** legal-consulting/, mortgage-service/, and property-sales-marketing/ also exist in the repo but are **not** in build.py's PAGES list and aren't linked from navigation - they're leftovers from a redesign that was reverted. See section 10.

2. The build system (build.py)

Instead of copy-pasting the header and footer into all twelve pages by hand, there is exactly ONE copy of each, in `partial/header.html` and `partial/footer.html`. Running `python build.py` stitches the current partial content into every page listed in PAGES, replacing whatever is between marker comments:

```
<!-- HEADER:START -->  ...generated, don't edit by hand...  <!-- HEADER:END -->
<!-- FOOTER:START -->  ...generated, don't edit by hand...  <!-- FOOTER:END -->
```

Every run **replaces** everything between those markers. A brand new page can start with a bare `<!-- HEADER -->` / `<!-- FOOTER -->` placeholder; the first build turns those into the marked blocks shown above.

The one rule that matters most

> **Note:** Page-specific <script> tags (enquiry.js, case-studies.js, booking.js, PapaParse, etc.) MUST be placed AFTER `<!-- FOOTER:END -->`. Anything placed between the FOOTER markers is silently deleted the next time build.py runs. This has caused real bugs during development - always double check this after adding a new script tag.

Template tokens

Inside the partials, these tokens get substituted per page:

- `{{ROOT}}` - empty string on the homepage, `../` on every other page, so relative links work from any folder depth.
- `{{NAV_*_ACTIVE}}` (one per nav item) - becomes `class="is-active"` on that page's own nav link, empty everywhere else.
- `{{NAV_GROUP_BUILD_ACTIVE}}`, `{{NAV_GROUP_MGMT_ACTIVE}}` - highlight a dropdown button when the current page belongs to that group.

To add a new page

- Create `new-page/index.html` containing bare `<!-- HEADER -->` and `<!-- FOOTER -->` placeholders.
- Add it to the PAGES list in build.py (and a matching NAV_KEYS entry + nav link in partial/header.html, if it should appear in the menu).
- Run `python build.py`.

3. Bilingual support (translations.js / i18n.js)

The whole site supports English and Traditional Chinese via a simple client-side toggle, no separate URLs or server-side locale routing.

How it works

- `translations.js` defines two flat objects, `en { key: "text" }` and `zh { key: "text" }`. It's a plain JS object literal (with comments), not JSON.
- Any element with `data-i18n="some_key"` gets its `textContent` replaced with that key's text in the active language.
- Add `data-i18n-html` alongside it if the text contains HTML tags (this sets innerHTML instead of textContent) - currently unused anywhere on the site, but supported.
- The toggle button (`#langToggle`) flips between languages and stores the choice in `localStorage` under the key `nexcon-lang`.

Keys used only from JavaScript

Some text (booking widget messages, error strings) is injected via a small helper `t(key, fallback)` inside booking.js/case-studies.js, rather than a `data-i18n` attribute in HTML. These keys still live in translations.js, they're just referenced differently. If you ever write a script to find "unused" translation keys by grepping for `data-i18n`, remember to also check for `t("key_name"` calls in the .js files, or you'll get false positives.

Adding a new translation key

- Add the key to **both** the en{} and zh{} objects in translations.js (same key name).
- Reference it with `data-i18n="your_key"` on the element, or `t("your_key", "fallback text")` from JS.
- The Chinese value can be left matching the English text temporarily, but keep both keys present - a key missing from one language silently falls back to showing nothing in that language's data-i18n elements (i18n.js skips elements whose key isn't found in the dictionary) or the fallback string for t() calls.

4. Styling and the cache-busting convention

All visual styling lives in one file, `styles.css`, using CSS custom properties for the brand palette (e.g. `var(--bordeaux)`, `var(--mahogany)`, `var(--sandstone)`).

Cache-busting: the ?vNN convention

Every page links the stylesheet as `styles.css?v=NN` (and booking.js as `booking.js?v=N` on the building-inspections page specifically). Browsers cache these files aggressively; without the version query string changing, a returning visitor can keep seeing a stale, previously-cached copy indefinitely after a CSS/JS update goes live.

> **Note:** Whenever you change styles.css, bump the `?v=` number on **every single page** that links it (all 12 live pages currently use the same number, so they must all be bumped together). Whenever you change booking.js, bump its own `?v=` number on building-inspections/index.html (the only page that loads it). As of this writing styles.css is at v=41 and booking.js is at v=9.

A quick way to check every page is on the same version before pushing:

```
grep -rn "styles.css?v=" --include="*.html" .
```

5. Contact form (enquiry.js + FormSubmit)

The enquiry/contact forms post to **FormSubmit** (https://formsubmit.co), a free third-party form-to-email relay that needs no backend or signup. enquiry.js handles this for every `form[data-enquiry]` on the page.

What it does

- Joins any ticked "project type" checkboxes into one hidden field before sending.
- Reveals a free-text box when a checkbox with a matching `data-detail` id is ticked, and appends that text to the joined value (e.g. "Other: solar installation").
- Strips non-digit characters from phone fields as the visitor types.
- Shows a thank-you message on success, or an error message if the request fails.

Spam protection - and its real limits

- A hidden honeypot field (`_honey`) and `_captcha=false` are sent to FormSubmit; FormSubmit's own filtering is the only server-side spam defense in place.
- A client-side daily send limit (15/day, tracked in `localStorage` under `nexconEnquiries`) exists purely as a courtesy against accidental repeat clicks - it is **not** real security. Clearing browser storage, using a private window, or scripting a direct POST to FormSubmit all bypass it trivially. The same pattern (and the same caveat) is reused for the booking widget's "3 bookings per browser" limit in section 7.

6. Case studies from a Google Sheet (case-studies.js)

The Projects page can show extra case studies pulled live from a published Google Sheet, so someone non-technical can add or edit a project write-up (in English and/or Chinese) without touching any code or asking a developer.

How it's wired

- A Google Sheet is published to the web as CSV (File > Share > Publish to web > CSV).
- That published CSV URL is hardcoded in case-studies.js as `SHEET_CSV_URL`.
- On page load, case-studies.js fetches and parses that CSV (using PapaParse, loaded from cdnjs only on the Projects page) and renders one card per row into `#sheet-projects`.

Sheet columns (row 1 headers, exact names)

```
title | title_zh | project | project_zh | location | location_zh | type | type_zh |
overview | overview_zh | features | features_zh | role | role_zh | challenges |
challenges_zh | result | result_zh | images | published
```

- Every English column has a matching _zh column for the Chinese translation of that field; leaving a _zh cell blank just falls back to showing the English text.
- "features"/"features_zh" and "images" can each hold multiple lines in one cell (Alt+Enter / Option+Return for a new line within the cell) - each line becomes one bullet point, or one image, respectively.
- "published" must be TRUE for a row to appear on the live site.
- Google Drive image links are automatically rewritten into Google's embeddable form; the Drive file must be shared as "Anyone with the link" or the image won't load for visitors.

> **Note:** Anyone with edit access to that Google Sheet can change what appears on the site, instantly, with no deploy step. The published CSV link itself is read-only to the outside world.

7. Inspection booking system

The building-inspections page has a real booking calendar backed by a free, serverless combination: a small frontend widget (booking.js), a Google Apps Script "Web App" acting as the only backend the site has, and a Google Sheet as the database.

Architecture

- **booking.js** (in this repo) - renders a month-view calendar, handles the booking form, and talks to the Apps Script over plain HTTP (fetch).
- **Google Apps Script** (NOT in this repo - lives only inside the Google Sheet's Extensions > Apps Script editor) - exposes `doGet` (read availability) and `doPost` (make a booking) as a deployed "Web App" URL. The full current source is reproduced in Appendix A so it isn't lost if access to the Google account is ever an issue - but the Apps Script editor is always the source of truth; treat Appendix A as a backup/reference, not something to copy from blindly without checking the live version first.
- **Google Sheet**, tab named exactly "Bookings", columns: `date | slot | name | email | phone | address | status | created`.

Rules enforced (both client-side for a responsive UI, and again server-side so nothing relies on a visitor's browser playing fair)

- A slot can only be booked at least 24 hours before it starts.
- A slot can't be booked more than 30 days ahead.
- No inspections on Saturday or Sunday.
- Two people can't double-book the same slot: the Apps Script uses `LockService.getScriptLock()` so a check-then-write is atomic across simultaneous requests.

The calendar UI

- A real month-grid calendar (Mon-Sun), not a scrolling list. Disabled/greyed days are either weekends, inside the 24-hour lead window, or beyond the 30-day limit.
- "Previous month" is disabled once you're back at the current month; "Next month" allows browsing exactly one month ahead of today (so visitors can see what's coming, even on days when the entire bookable window sits inside the current calendar month) but no further.
- Clicking an enabled day reveals its AM (10:00-1:00) and PM (2:00-5:00) slots beneath the calendar; picking an open slot opens the booking form (name, email, phone, property address).
- After a booking succeeds, a race-condition warning appears, or nothing changes for other slots - the widget never re-renders the whole calendar in one action, since that would wipe out whatever success/warning message was just shown before anyone could read it. Instead each slot button has a stable id (`slot-YYYY-MM-DD-AM`) and only that one button is swapped between "open" and "taken".

The 3-bookings-per-browser limit

At most 3 active bookings are allowed from one browser, tracked in `localStorage` under `nexconMyBookings` - the same pattern used for the contact form's daily limit. It's a courtesy limit only, not a real security control: clearing site data or using a different browser bypasses it immediately. Because a booking can also be cancelled by staff directly in the Sheet (see below), the widget reconciles this local list against the live Sheet on every page load/refresh, dropping any locally-tracked booking the Sheet no longer shows as booked.

Cancellations: email the business, no self-service link

An earlier version of this feature emailed customers a one-time "click to cancel" link. It was removed after testing showed it was unreliable for real customers: some email apps (notably Outlook) render links from an email inside their own embedded frame rather than a normal browser tab, and Google's pages explicitly refuse to load inside a foreign frame - producing a confusing "refused to connect" or Drive-style error for exactly the people trying to cancel.

Cancellation is now fully manual and intentionally simple:

- The booking confirmation email and the on-site note both tell the customer to email the business (building@nexcongroup.com.au) at least 24 hours beforehand to cancel or reschedule.
- Staff then open the Bookings Sheet, find that row, and change its "status" cell from `booked` to `cancelled`. That single edit is enough - the next time anyone loads the booking page, that slot shows as available again.

One-time setup, if this ever needs to be rebuilt from scratch

- Create a Google Sheet with a tab named exactly "Bookings" and the header row described above.
- Paste the Apps Script source (Appendix A) into that Sheet's Extensions > Apps Script editor.
- Deploy it as a Web App: Execute as "Me", Who has access "Anyone".
- Paste the resulting deployment URL into booking.js as `WEBAPP_URL`.

Redeploying after an Apps Script code change

- Edit the code directly in the Apps Script editor (there is no local copy to edit-and-push).
- Deploy > Manage deployments > pencil/edit icon on the existing deployment > Version: "New version" > Deploy. This keeps the same URL live, so booking.js never needs to change.
- Always re-test afterward: GET the Web App URL directly (should return `{"ok":true,"booked":[...]}`) and, if practical, do one real test booking end-to-end.

8. Security posture and known gaps

A code-level audit of the whole site (as of this writing) found no actual injection vulnerabilities. Summary of what was checked and what remains open:

In good shape

- No `eval`, `document.write`, or unescaped `innerHTML` anywhere. Every dynamic HTML write (booking widget, case-studies pulled from the public Sheet, translations) is passed through an `esc()` helper (a textContent round-trip) before being inserted.
- Case-study image URLs are restricted to http/https only (blocks `javascript:` URLs).
- No hardcoded secrets, API keys, or credential files anywhere in the repository. The Apps Script Web App URL being public in booking.js is intentional, not an oversight - it's designed to be called directly.
- No `target="_blank"` links anywhere (so no reverse-tabnabbing risk) and no plain-`http://` resource loads.

Worth doing (low effort, no downside)

- No Content-Security-Policy meta tag anywhere in <head>. Doesn't fix anything today, but is free defense-in-depth if an XSS vector is ever introduced later (e.g. via a third-party script).
- PapaParse, loaded from cdnjs.cloudflare.com on the Projects page only, has no `integrity` (SRI) attribute. If cdnjs were ever compromised, that script could be swapped for something malicious without any warning.

Accepted limitations (not bugs, just know they exist)

- Both the contact form and the booking widget's "per-browser" limits are client-side only (localStorage) and trivially bypassed by anyone willing to clear storage or call the endpoints directly - acceptable risk for a small local business site, but not a real rate limit.
- Neither the FormSubmit contact endpoint nor the Apps Script booking endpoint has a CAPTCHA or server-side rate limit, so a determined script could create many fake bookings/enquiries. FormSubmit's own spam filtering is the only backstop on that side; the Apps Script side has none beyond the booking rules themselves (24h/30-day/weekday/no-double-booking).

Operational checks (not code, but worth confirming periodically)

- The Bookings Google Sheet holds customer names, emails, phones, and addresses - confirm it is shared only with the business's own Google account(s), never "anyone with the link."
- Confirm GitHub Pages has "Enforce HTTPS" turned on for the custom domain, under the repository's Settings > Pages.

9. Deployment and day-to-day workflow

There is no CI/build pipeline. GitHub Pages serves the repository's main branch directly; pushing to main is the deploy step, and changes typically appear live within a minute or two.

Before pushing a change, checklist

- If you edited partial/header.html or partial/footer.html: run `python build.py` and confirm every page printed "Updated" (or "Unchanged" if nothing actually changed for that page).
- If you edited styles.css: bump the `?v=` number on every page that links it (see section 4).
- If you edited booking.js: bump its `?v=` number on building-inspections/index.html.
- Preview locally first: `python -m http.server 8000` from the repo root, then open `http://localhost:8000/` in a browser.
- Double-check any new page-specific <script> tag is placed AFTER `<!-- FOOTER:END -->`, not before.

Hosting details

- Custom domain: nexconbm.com.au, set via the CNAME file at the repo root.
- robots.txt explicitly allows normal search engines and "AI search/assistant" crawlers that fetch pages to answer a user's question, while disallowing bots that scrape content to train AI models.

10. Known loose ends

- **Orphaned redesign pages:** legal-consulting/, mortgage-service/, and property-sales-marketing/ (plus a couple of unused .webp images) are leftovers from a site redesign that was started and then reverted. They aren't in build.py's PAGES list and aren't linked from navigation or the sitemap, so they're invisible to visitors, but they still exist in the repo. Decide whether to finish them, delete them, or leave them as-is.
- **CSP meta tag and SRI hash for PapaParse** - see section 8, low-effort hardening not yet done.
- **Google Sheet sharing settings** for the Bookings sheet and the case-studies sheet should be spot-checked periodically (see section 8).

Appendix A: full Apps Script backend source

This is the complete current source of the booking system's backend, as deployed in the Google Apps Script editor (Extensions > Apps Script, inside the Bookings Google Sheet). It does not live in this git repository - this appendix exists purely so the code isn't lost if access to that Google account is ever lost or handed over. **Always treat the live Apps Script editor as the source of truth**; this is a snapshot, not a synced copy.

```
/* Nexcon inspection booking backend.
   The Sheet needs one tab named exactly "Bookings" with this header row (row 1),
   in this exact order:
     date | slot | name | email | phone | address | status | created

   "date" is stored as YYYY-MM-DD text, "slot" is "AM" or "PM", "status" is "booked" or
   "cancelled". Don't type anything into those columns yourself -- the script fills them
   in as people book.

   There's no self-service cancellation here -- that was tried via a one-time link in the
   confirmation email, but turned out unreliable in practice: some email apps (notably
   Outlook) render emailed links inside their own embedded frame, and Google's pages
   refuse to load inside a foreign frame, breaking the link for exactly the people who
   needed it. Instead, the booking confirmation email just asks the customer to email the
   business to cancel; staff then cancel it manually by changing that row's "status" cell
   from "booked" to "cancelled" directly in this Sheet. */

const SHEET_NAME = 'Bookings';
const NOTIFY_EMAIL = 'building@nexcongroup.com.au'; // where booking emails go
const SLOT_LABELS = { AM: '10:00 AM - 1:00 PM', PM: '2:00 PM - 5:00 PM' };
const MAX_DAYS_AHEAD = 30;

function getSheet_() {
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName(SHEET_NAME);
  if (!sheet) throw new Error('No sheet tab named "' + SHEET_NAME + '" was found.');
  return sheet;
}

function columnIndex_(header) {
  const map = {};
  header.forEach((h, i) => { map[h] = i; });
  return map;
}

// Google Sheets silently converts a "2026-10-03"-shaped string into its own Date value
// when a row is written, unless the cell is explicitly forced to stay plain text. Once
// that happens, a value read back is a Date object, not the original string, so a plain
// "===" comparison against an incoming date string would never match -- which is exactly
// what broke the "is this slot already taken" check. This normalizes either shape back to
// a plain YYYY-MM-DD string before anything compares dates, so it's safe either way.
function normalizeDate_(value) {
  if (value instanceof Date) {
    return value.getFullYear() + '-' + String(value.getMonth() + 1).padStart(2, '0') + '-' + String(value.getDate()).
  }
  return String(value).trim();
}

function json_(obj) {
  return ContentService.createTextOutput(JSON.stringify(obj)).setMimeType(ContentService.MimeType.JSON);
}

// GET: returns which date+slot combinations are currently booked, so the site can grey
// them out before anyone tries to click one.
function doGet(e) {
  const sheet = getSheet_();
  const rows = sheet.getDataRange().getValues();
  const col = columnIndex_(rows[0]);
  const booked = [];
  for (let i = 1; i < rows.length; i++) {
    if (rows[i][col.status] === 'booked') {
      booked.push(normalizeDate_(rows[i][col.date]) + '_' + rows[i][col.slot]);
    }
  }
```

```
  return json_({ ok: true, booked: booked });
}

// POST: makes a new booking. Uses a script-wide lock so two people clicking "book" on
// the same slot at the same moment can't both succeed -- whichever request gets the lock
// first checks and writes; the second sees it's already taken.
function doPost(e) {
  const lock = LockService.getScriptLock();
  try {
    lock.waitLock(10000);
  } catch (err) {
    return json_({ ok: false, error: 'busy' });
  }
  try {
    const data = JSON.parse(e.postData.contents);
    if (data.action === 'book') return handleBook_(data);
    return json_({ ok: false, error: 'unknown_action' });
  } catch (err) {
    return json_({ ok: false, error: 'server_error' });
  } finally {
    lock.releaseLock();
  }
}

function handleBook_(data) {
  const { date, slot, name, email, phone, address } = data;
  if (!date || !slot || !name || !email || !phone || !address) {
    return json_({ ok: false, error: 'missing_fields' });
  }
  if (SLOT_LABELS[slot] === undefined) return json_({ ok: false, error: 'bad_slot' });

  const slotStart = slotStartDate_(date, slot);
  const dow = slotStart.getDay(); // 0 = Sun, 6 = Sat
  if (dow === 0 || dow === 6) return json_({ ok: false, error: 'weekend' });
  const now = new Date();
  const hoursUntil = (slotStart - now) / 3600000;
  if (hoursUntil < 24) return json_({ ok: false, error: 'too_soon' });
  const maxDate = new Date(now.getTime() + MAX_DAYS_AHEAD * 24 * 3600000);
  if (slotStart > maxDate) return json_({ ok: false, error: 'too_far' });

  const sheet = getSheet_();
  const rows = sheet.getDataRange().getValues();
  const col = columnIndex_(rows[0]);

  for (let i = 1; i < rows.length; i++) {
    if (normalizeDate_(rows[i][col.date]) === date && rows[i][col.slot] === slot && rows[i][col.status] === 'booked')
      return json_({ ok: false, error: 'taken' });
    }
  }

  sheet.appendRow([date, slot, name, email, phone, address, 'booked', new Date()]);
  // Force the date cell we just wrote to stay plain text, so it reads back exactly as
  // typed rather than being silently reinterpreted as a Sheets date.
  const newRow = sheet.getLastRow();
  sheet.getRange(newRow, col.date + 1).setNumberFormat('@').setValue(date);

  const slotLabel = SLOT_LABELS[slot];

  MailApp.sendEmail(NOTIFY_EMAIL,
    'New inspection booking: ' + date + ' (' + slotLabel + ')',
    'Name: ' + name + '\nEmail: ' + email + '\nPhone: ' + phone + '\nAddress: ' + address +
    '\nDate: ' + date + '\nTime: ' + slotLabel);
  MailApp.sendEmail(email,
    'Your Nexcon inspection booking is confirmed',
    'Hi ' + name + ',\n\nYour building inspection is booked for:\n' + date + ', ' + slotLabel +
    '\nAddress: ' + address +
    '\n\nNeed to cancel or reschedule? Just reply to this email or call us, at least 24 ' +
    'hours before the appointment.' +
    '\n\nQuestions? Call +61 411 330 962 or reply to this email.' +
    '\n\nThanks,\nNexcon Building Management');

  return json_({ ok: true });
}
function slotStartDate_(dateStr, slot) {
```

```
  const [y, m, d] = dateStr.split('-').map(Number);
  const hour = slot === 'AM' ? 10 : 14;
  return new Date(y, m - 1, d, hour, 0, 0);
}
```